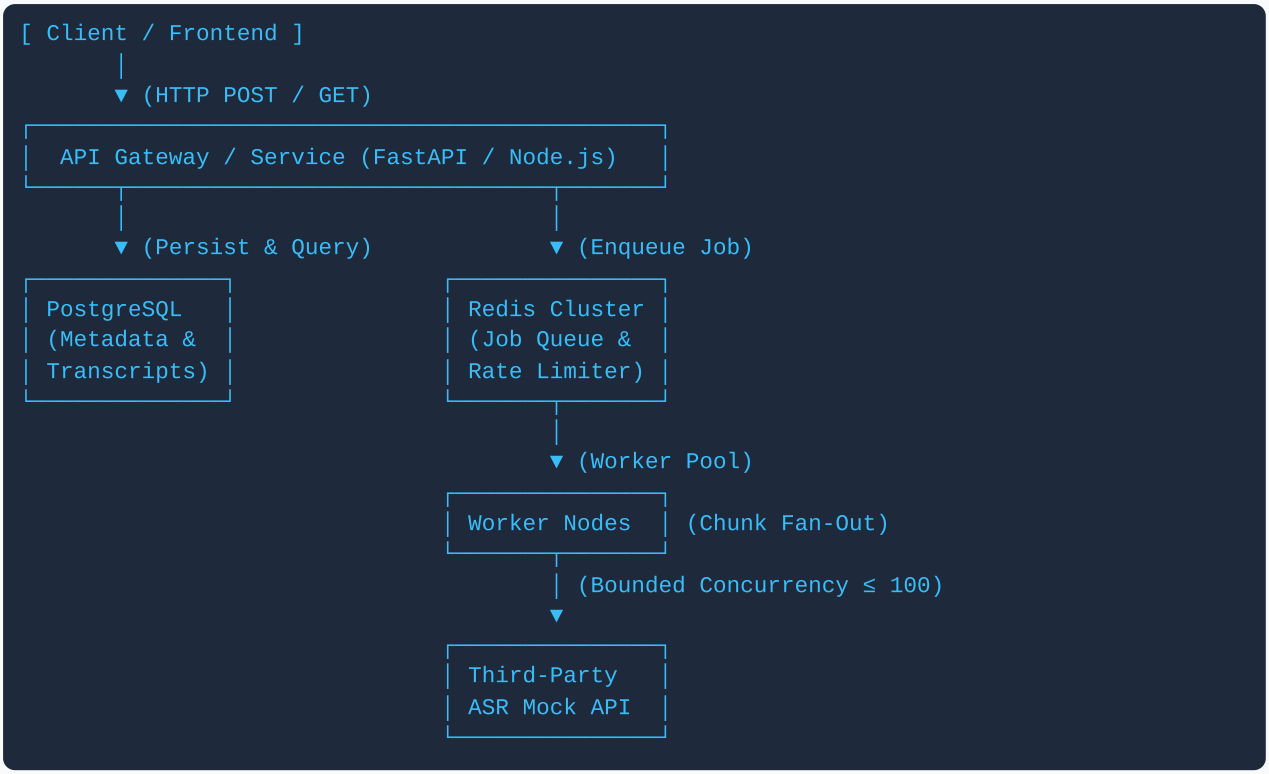
SYSTEM DESIGN SPECIFICATION

Backend Audio Transcription Service

1. Executive Summary & Architecture

This document details the production design for an asynchronous, high-throughput backend service designed to process multi-chunk patient/provider audio recordings and stitch them into unified transcripts. The service strictly enforces a **100-request ASR model concurrency cap** while guaranteeing sub-20s happy-path latency.

System Architecture Overview



2. API Specifications & Data Contracts

POST /transcribe

```
// Request Payload
{
  "audioChunkPaths": ["s3://bucket/audio_1.wav", "s3://bucket/audio_2.wav"],
  "userId": "user_12345"
}

// Response (201 Created)
{ "jobId": "job_abc123xyz" }
```

GET /transcript/{jobId}

```
// Response (200 OK)
{
  "jobId": "job_abc123xyz",
  "jobStatus": "COMPLETED",
  "completedTime": "2026-10-21T17:45:00Z",
```

3. Core Engineering Solutions

A. Concurrency Limiting (≤ 100)

Managed via a Redis Distributed Semaphore (`ASR_SEMAPHORE`). Workers check and increment before dispatching HTTP calls. If count > 100, workers back off for 100–250ms. Counter decrements immediately upon ASR call completion.

B. Sub-20s Latency Optimization

Workers launch parallel async greenlets/coroutines for all chunks in a job simultaneously. For a 30-min encounter (6 chunks), latency is bounded by a single 5–10s chunk duration rather than cumulative serial execution.

C. Fault Tolerance & Flakiness Handling (5% Transient Failure Rate)

PostgreSQL maintains authoritative job state for seamless server restart recovery. Transient ASR failures utilize 3x retries with exponential backoff and randomized jitter (1s, 2s, 4s). Unrecoverable audio segments mark job as `PARTIAL_SUCCESS`, stitching available segments with explicit missing-audio placeholders.

4. Napkin Math & System Sizing

Workload & Capacity Equations

- **Initial Load (1,000 DAU)**: 1,000 users × 10 visits/shift = 10,000 visits/day (60,000 chunks/day).
- **Daily Storage Growth**: 10,000 visits × 25 KB transcript text = **250 MB / day** (~90 GB / year).
- **ASR Throughput Limit**: 100 max concurrency ÷ 7.5s avg latency = **13.33 chunks / sec**.
- **Max DAU under 100 Limit**: 13.33 chunks/s × 28,800s shift = 384k chunks/day = 64k visits/day. Applying a 2.0 peak-hour clustering factor yields a maximum ceiling of **3,200 DAU** before queuing occurs.
- **10x Target (10,000 DAU)**: 100k visits/day = 600k chunks/day. Peak traffic demands ~1,500 concurrent ASR calls.

5. 10x Scalability Strategy (1,000 → 10,000 DAU)

System Component	Initial Version (1,000 DAU)	10x Scale Version (10,000 DAU)
ASR Concurrency	100 limit (Vendor default)	300–500+ limit (Vendor Upgrade + Redis Priority Queue)
Compute / Worker Pool	2 static worker nodes	Kubernetes Pods with Horizontal Pod Autoscaler (HPA on queue depth)
Database Layer	Single `db.m6g.large` instance	`db.m6g.xlarge` + PgBouncer Pooler + Read Replica + Monthly Partitioning
Task Queue / Cache	Standalone Redis instance	Sharded Redis Cluster with rate-limited token leasing
Search Optimization	Direct SQL B-Tree indexes	Compound Indexes (`user_id`, `job_status`) + S3 Cold Storage Archival